

RESEARCH ARTICLE

Permission-Aware RAG: Identity and Access Management (IAM)-Based Access Filtering in Multi-Resource Environments

JOOYOUNG JEONG¹ AND SANG-GOO LEE^{2,3}, (Member, IEEE)¹Graduate School of Engineering Practice, Seoul National University, Seoul 08826, Republic of Korea²Department of Computer Science and Engineering, Seoul National University, Seoul 08826, South Korea³IntelliSys Company Ltd., Seoul 08826, South Korea

Corresponding author: Sang-Goo Lee (sglee@europa.snu.ac.kr)

ABSTRACT Retrieval-Augmented Generation (RAG) enhances large language models (LLMs) by integrating external knowledge sources, which significantly improves response accuracy and contextual relevance. However, most existing RAG research implicitly assumes that all retrieved content is equally accessible to any user. This approach fails to address the complex, fine-grained access control required in enterprise environments in which data originates from multiple, independently governed sources. Attempting to merge heterogeneous authorization models, such as RBAC and ABAC, often leads to significant compliance burdens and unintended data exposure. To address this issue, we propose a permission-aware RAG framework that enforces resource-level access control by directly interfacing with provider-controlled Identity and Access Management (IAM) systems. By performing real-time permission validation against native IAM endpoints without requiring policy merging, this design preserves governance boundaries and ensures compliance. We implemented a prototype and conducted a comprehensive evaluation using a multi-hop QA benchmark. The evaluation included scenario-based qualitative analysis to verify policy enforcement, large-scale quantitative validation that confirms its scalable policy enforcement and quantifies the impact of access levels on answer quality. Results demonstrate that our framework provides secure, scalable information retrieval while maintaining strict adherence to diverse access policies across multi-resource environments.

INDEX TERMS Access control, identity and access management (IAM), large language models (LLMs), multi-resource environments, permission-aware retrieval, retrieval-augmented generation (RAG), secure information retrieval.

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities in a wide range of natural language tasks, leading to significant advances in AI applications [1]. However, when deployed in domain-specific or high-stakes environments, LLMs face notable limitations, including in fields such as healthcare [2], [3], law [4], and finance [5]. In particular, LLMs lack access to up-to-date or institution-specific knowledge at inference time, undermining their reliability and effectiveness for critical decision-making [6]. To

mitigate this gap, Retrieval-Augmented Generation (RAG) has emerged as a promising approach that enhances LLMs by injecting relevant external documents retrieved from structured or unstructured knowledge sources directly into the generation pipeline [7].

Most existing RAG research implicitly assumes a uniform access model, where all retrieved content is equally accessible to any user of the system [6], [8]. This implicit-trust approach is fundamentally incompatible with the modern principles of a Zero Trust Architecture (ZTA) [9], which mandate fine-grained, per-resource authorization based on the principle of least privilege. This assumption fails to address the access control challenges that arise in multi-organizational or

The associate editor coordinating the review of this manuscript and approving it for publication was Sedat Akleyek¹.

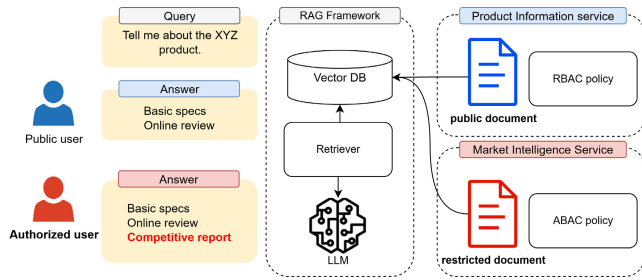


FIGURE 1. Access control needs in RAG systems. The same query generates different responses depending on the user's authorization level.

multi-service environments, where data originates from sources governed by distinct identity and policy frameworks. As a result, these systems fall short in addressing real-world scenarios where fine-grained, user-specific authorization is required to ensure privacy, security, and regulatory compliance.

As illustrated in Fig. 1, the same query can produce different responses depending on the user's access level. For example, public users may be limited to viewing basic product specifications and publicly available reviews, whereas authorized employees might retrieve confidential internal reports or proprietary analyses. This scenario highlights the necessity for enforcing user-specific access control in RAG systems, especially in multi-source environments where the sensitivity and ownership of data vary across sources.

Current RAG architectures lack mechanisms necessary for fine-grained, context-aware access control. When a centralized vector database aggregates documents from multiple data providers, applying a unified access policy across all content becomes both operationally complex and legally impractical. Centralized attempts to merge policies often result in permission escalation, ambiguous policy interpretation, and significant compliance overhead. Furthermore, integrating heterogeneous access control models, such as Role-Based Access Control (RBAC) [10] and Attribute-Based Access Control (ABAC) [11], into a single enforcement layer can introduce semantic inconsistencies and lead to unintended data leakage.

To address these challenges, we propose a *Permission-Aware RAG* framework that enables fine-grained access control across multi-resource environments. Our approach preserves centralized retrieval efficiency while delegating access control to Identity and Access Management (IAM) systems operated by each provider. Instead of attempting to merge or reinterpret access policies, the framework dynamically verifies document-level permissions at query time by interfacing with native IAM endpoints such as Google Cloud Platform (GCP) IAM, Amazon Web Services (AWS) IAM, or open-source Keycloak. By leveraging existing IAM infrastructures that are already trusted and maintained by each organization, our approach ensures policy consistency and practical deployability. This decentralized control model reduces legal risks, respects policy

autonomy, and ensures compatibility with diverse security architectures.

The key contributions of this paper are as follows:

- We propose a novel RAG framework that supports permission-aware retrieval across multi-resource environments by enabling fine-grained access control across heterogeneous policy domains.
- We implemented a prototype integrated with GCP, AWS, and Keycloak IAM, validating the feasibility of our approach in real-world environments.
- We validate the framework through three evaluations on the HotpotQA dataset: (i) scenario-based qualitative evaluation, (ii) large-scale quantitative evaluation, and (iii) latency impact evaluation.

II. RELATED WORK

A. ACCESS CONTROL NEEDS IN RAG SYSTEMS

RAG systems are increasingly used in multi-resource environments where knowledge is distributed across diverse data sources. For example, in many enterprises, data resources are distributed across multiple departments, services, and infrastructure environments. In healthcare organizations, patient records may be stored separately within hospitals, clinics, and research labs due to privacy policies, organizational boundaries, and system silos. In finance, investment reports, client data, and regulatory documents are often managed using different cloud providers, on-premise servers, or SaaS platforms depending on departmental budgets, policies, and compliance requirements. Such fragmented environments make centralized data integration challenging and result in complex multi-resource settings that RAG systems must operate within [12].

Each data source is governed by its own access control policies and regulatory requirements. Healthcare data must comply with HIPAA [13], GDPR [14], and local privacy laws. Financial data is subject to strict industry-specific regulations, and government documents often require strict security clearances. Furthermore, these policies are implemented using heterogeneous models such as RBAC or ABAC, making unified enforcement complex. Without aligning retrieval processes with these diverse policies, RAG systems risk unauthorized disclosure, policy conflicts, and regulatory violations [15], [16].

Therefore, RAG systems require fine-grained (resource-specific), user-specific, and real-time access validation mechanisms. However, existing RAG frameworks lack built-in capabilities to perform such dynamic authorization at query time. The need for such mechanisms has been formally recognized in recent security analyses of RAG systems. For instance, a systematic review by Ammann et al. identifies *Retrieval Data Leakage* as a primary threat and proposes *Access Limitation* as a critical mitigation strategy [17]. This underscores the academic and practical importance of developing robust access control frameworks specifically for RAG. Therefore, a central challenge in RAG security research

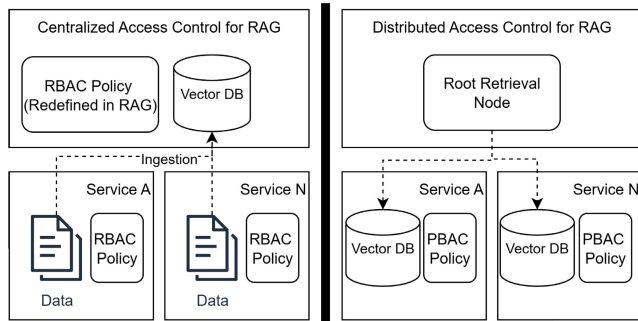


FIGURE 2. Comparison between centralized and distributed access control models for RAG.

is to determine how to effectively implement this *Access Limitation* in complex, real-world enterprise environments.

B. REPRESENTATIVE APPROACHES TO INTEGRATED ACCESS CONTROL

Prior research has proposed multiple strategies for integrating access control into RAG systems. As illustrated in Fig. 2, these approaches can be broadly categorized into centralized and distributed architectures, depending on how documents are stored and policies enforced.

Centralized frameworks consolidate documents from various sources into a single vector database and apply internal filtering mechanisms. For example, Chen et al. [18] proposed a proof-of-concept implementation that maintains a centralized vector database alongside an internal permission store. A more structured centralized approach is seen in VAULT, which models access control rules directly within a Knowledge Graph [19]. However, this model necessitates a complex and continuous synchronization of its internal node-level access attributes with the original IAM systems, creating a significant challenge for maintaining real-time policy consistency, especially in heterogeneous environments.

While these different architectures enable streamlined management within a single policy domain, they share a critical limitation: they require merging diverse access control models, such as RBAC [10] and ABAC [11], into a unified enforcement layer. This process is not only operationally burdensome but also introduces semantic mismatches and risks of unintended data leakage or privilege escalation due to flawed policy mapping.

Distributed frameworks, by contrast, maintain policy autonomy by enforcing access control within each data provider's environment. For instance, MediRAG [20] implements a federated retrieval system across hospitals, using policy-based access control (PBAC) [11] at each node to ensure that organizations can independently define and enforce policies. This architecture preserves data sovereignty and aligns well with compliance requirements in sensitive domains like healthcare. However, MediRAG assumes that all nodes adopt a uniform PBAC model, which is often unrealistic. Real-world enterprises frequently employ

heterogeneous IAM systems such as GCP IAM, AWS IAM, or Keycloak, each with its own authentication workflows, policy languages, and role hierarchies. Requiring standardization across such diverse systems introduces substantial adoption barriers. Furthermore, MediRAG lacks integration with these native IAM systems, making centralized governance and cross-domain identity management difficult to scale. We address this gap by delegating authorization to each provider's native IAM at retrieval time, rather than consolidating policy management.

In environments where heterogeneous access control models such as RBAC, ABAC, and custom policies coexist across administrative domains, requiring centralized policy consolidation becomes highly impractical. Centralized RAG architectures struggle with the complexity and risk of merging diverse policies, while distributed frameworks often rely on a unified access control model that does not reflect real-world diversity. These limitations underscore the need for an alternative approach that can respect policy autonomy while still enabling efficient retrieval. To this end, we propose Permission-Aware RAG, a framework that supports fine-grained access control through real-time permission validation by directly interfacing with native IAM systems. This design enables secure and scalable retrieval across heterogeneous multi-resource environments without requiring structural reconfiguration.

C. FEDERATED IDENTITY AND ZERO TRUST ARCHITECTURES

While *Federated Identity Management (Federated IAM)* provides the foundational layer for authentication in multi-service environments, recent surveys highlight persistent challenges such as privacy risks, attribute aggregation, and revocation [21], [22]. Privacy-preserving extensions like *PRIMA* enable selective disclosure and unlinkability between identity providers (IdPs) and service providers (SPs) [23]. Reference models such as the *Reference Service Model Framework for Identity Management* attempt to standardize identity management architectures across centralized, federated, and user-centric models [24]. However, none of these approaches address the subsequent, more granular challenge of real-time, per-document authorization within retrieval-augmented systems.

Our Permission-Aware RAG framework is designed to solve precisely this challenge. It operates on the principles of a *Zero Trust Architecture (ZTA)* [9], which mandates continuous verification for every access request. While Federated IAM establishes *who* the user is, our framework determines *what* that authenticated user is allowed to see at the moment of retrieval. By delegating this check to native IAM systems, we enforce ZTA's *least privilege* principle for every retrieved resource, bridging a critical gap between federated authentication and practical, secure, access-controlled retrieval in modern enterprise environments. This approach is also compatible with existing enterprise Single Sign-On

(SSO) mechanisms, as it leverages the same IAM endpoints used for federated authentication.

III. METHODOLOGY

A. MOTIVATION FOR IAM-BASED INTEGRATION

Modern enterprises use IAM systems such as AWS IAM, GCP IAM, Azure Active Directory (Azure AD), and Keycloak to manage fine-grained access control across their resources. These systems provide authentication, authorization, RBAC/ABAC policy support, audit logging, and version-controlled policy management to ensure security and regulatory compliance. For example, depending on service requirements, resources may be deployed across GCP, AWS, or on-premise environments. Organizational policies may mandate that certain data be processed exclusively on-premise for compliance reasons, while other services leverage cloud-based solutions.

Although consolidating diverse access control policies into a single, unified policy across all services and organizations may be considered ideal, achieving such integration in practice requires extensive legal reviews, cross-institutional policy reconciliations, and thorough conflict analysis across heterogeneous access models [25]. These processes, such as policy mapping and role harmonization, are time-consuming and operationally burdensome, often delaying the deployment of Retrieval-Augmented Generation (RAG) systems that require timely integration with external knowledge sources.

To address this challenge, our framework integrates RAG systems with existing IAM infrastructures, enabling real-time access validation without requiring full policy unification. This design maintains the independent policy governance of each organization while ensuring compliance, operational feasibility, and immediate deployability. By leveraging the trusted IAM endpoints operated by each provider, the system maintains consistency with institutional policy structures (e.g., RBAC, ABAC, resource policies) and prevents semantic mismatches. It also integrates seamlessly with native IAM functionalities such as audit logging, policy evaluation, and token validation, thereby minimizing legal and compliance risks in multi-resource environments.

B. THREAT MODEL

Our framework's security model is grounded in the *Zero Trust Architecture (ZTA)* principle of *never trust, always verify*, treating every document access request as a distinct transaction that requires explicit authorization [9]. Based on this, we use the *STRIDE* model to define our scope [26].

- **Information Disclosure (In Scope):** This is the primary threat our framework prevents. Conventional RAG systems implicitly trust any retrieved document, creating a significant risk of leaking sensitive information. Our framework mitigates this by enforcing real-time, per-document authorization against native IAMs.

- **Elevation of Privilege (In Scope):** We address unintentional privilege escalation at the data access layer. By delegating authorization checks to the authoritative source (the native IAM), our framework prevents flawed logic, such as faulty policy merging, from granting users access to data beyond their intended scope.
- **Spoofing, Tampering, Repudiation, and Denial of Service (Out of Scope):** Our framework focuses specifically on authorization and assumes other foundational security layers are in place. We assume user identity verification (spoofing) is handled by upstream authentication systems (e.g., SSO). Threats to data integrity (tampering), auditability (repudiation), and service availability (denial of service) are considered out of scope, to be addressed by underlying infrastructure security.

C. SYSTEM ARCHITECTURE

The Permission-Aware RAG framework is designed to enable access control across multi-resource environments without requiring central policy merging. As illustrated in Fig. 3, the architecture's modular pipeline is centered around the Permission Retriever, which utilizes a Credential Manager and an IAM Adapter to filter documents retrieved from the Vector Store, using details from the Metadata Store. This design ensures both maintainability and scalability.

Each module addresses specific operational needs. For example, by separating the Credential Manager from the IAM Adapter, security credentials can be managed and updated independently of permission validation logic, ensuring flexibility and security.

To enforce user-specific access rights, document permissions are delegated to the IAM systems operated by each service provider. This design preserves the independent policy governance of each organization while enabling real-time authorization checks without requiring unified policy consolidation. By avoiding the need to merge policies, the framework mitigates risks of semantic conflicts and ensures compatibility with heterogeneous access control models deployed by different institutions.

The system operates in two primary stages: the Indexing Stage and the Query Stage, described below.

1) INDEXING STAGE

- (a) Data are collected from each service or organization and sent to the Data Ingestion Manager.
- (b) The Data Ingestion Manager assigns a globally unique `resourceID` to each resource, serving as a persistent identifier linking document chunks to their provider and IAM information, while also enabling efficient metadata updates and deletions when resource information changes over time.
- (c) Document chunks, their corresponding embeddings, and the associated `resourceID` are stored in the vector store as `vector_meta`, referring to embedding

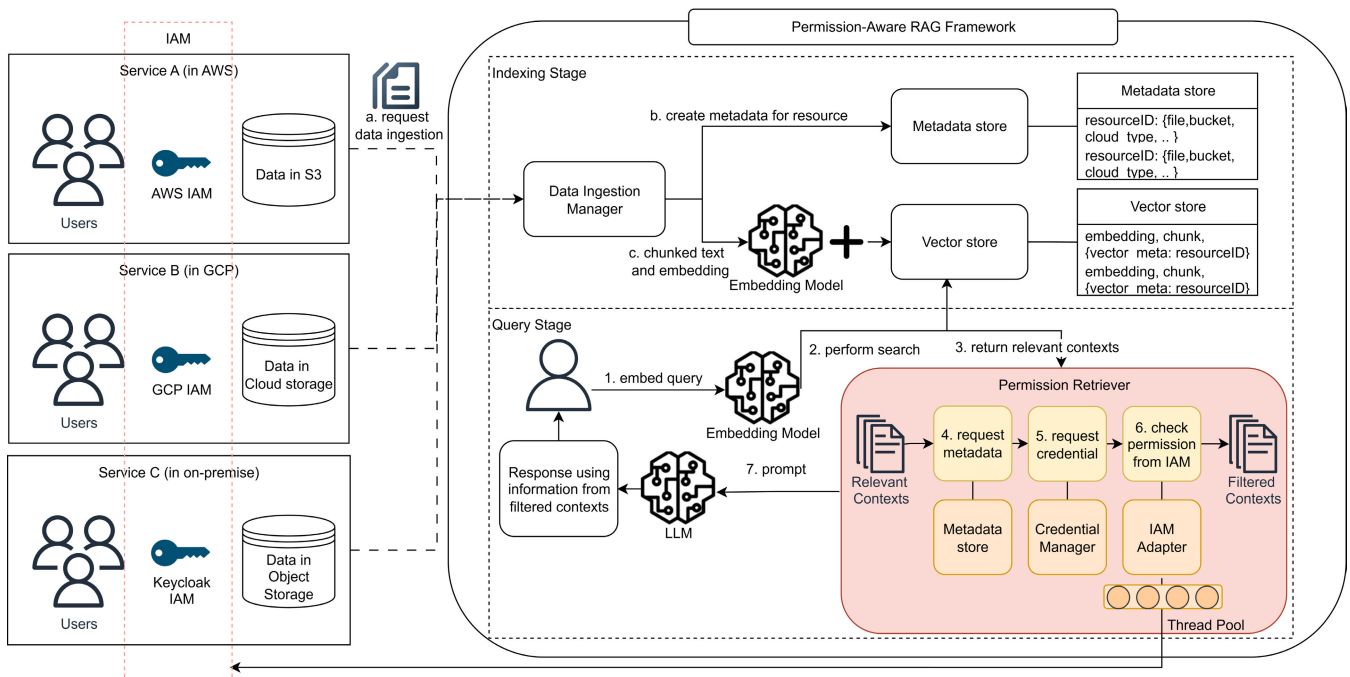


FIGURE 3. Overall architecture of the proposed Permission-Aware RAG system.

records coupled with resource metadata for downstream retrieval and filtering.

This indexing stage normalizes and stores all metadata and embedding information required for retrieval and access control. Its successful execution determines real-time search speed and validation efficiency in the subsequent query stage, enabling fast and accurate Permission-Aware RAG responses even in large-scale environments containing millions of documents.

2) QUERY STAGE

- (1) The user submits a query, which is encoded via an embedding model.
- (2) The vector store is searched to identify semantically relevant documents.
- (3) The top- k relevant documents are passed to the Permission Retriever module.
- (4) For each document, the `resourceID` stored in `vector_meta` is used to retrieve its IAM metadata from the Metadata Store.
- (5) The Credential Manager retrieves necessary credentials required for authorization checks.
- (6) The IAM Adapter receives permission requests for relevant documents and concurrently verifies them with IAM services. It then returns the validation results, enabling the system to filter out unauthorized documents.
- (7) Filtered authorized documents and the user query are passed to the LLM to generate the final answer.

The architecture is designed to support horizontal scaling of the Permission Retriever and IAM Adapter modules, which is expected to handle high query volumes with minimal

performance degradation. By delegating access validation to native IAM systems rather than consolidating policy management, the framework mitigates risks such as permission misconfiguration (incorrect access settings granting unintended permissions), policy drift (gradual divergence between intended and actual policies over time), and privilege escalation (users gaining higher privileges than originally authorized). This decentralized validation model ensures that each data provider retains full control over resource access decisions, aligning with security and compliance requirements.

D. CORE MODULES

1) DATA INGESTION

To provide a scalable and reliable entry point for all external resources, the Data Ingestion module is responsible for collecting and ingesting resources from diverse data sources into the Permission-Aware RAG framework. Each ingested resource is assigned a globally unique identifier called `resourceID`, generated as a UUID, ensuring persistent and collision-free identification across the system. Simultaneously, the module stores the original data source identifier as `local_resource_id`, which references the resource's native identifier within its originating platform or repository.

This dual-identifier design significantly enhances traceability by maintaining both a system-wide unique reference (`resourceID`) and the source-specific identifier (`local_resource_id`). Such structure enables reliable CRUD operations, ensuring that modifications or deletions can be efficiently propagated back to the original data source while preserving consistent mappings within the RAG system.

The detailed metadata schema for each resource, including provider-specific parameters required for permission validation, is provided in Appendix A.

A key design principle is the decoupling of static vector embeddings from dynamic metadata. The vector store holds embeddings linked by resourceID for rapid retrieval, while the metadata store contains information prone to operational changes, such as file names and storage locations. This separation is crucial for maintainability and efficiency, as it allows metadata to be updated independently without the costly re-indexing of vector embeddings.

In summary, the Data Ingestion module combines a dual-identifier mechanism for robust traceability with a decoupled storage architecture for operational efficiency. This approach provides a scalable and maintainable foundation, ensuring that permission-aware retrieval can function effectively in complex, dynamic enterprise environments.

2) CREDENTIAL MANAGER

The Credential Manager module serves as an orchestration gateway for obtaining, managing, and providing service-specific credentials required for authorization checks across heterogeneous platforms such as GCP, AWS, and Keycloak. Its primary purpose in production environments is to integrate with external Identity Providers (IdPs) such as Active Directory, Okta, or enterprise Single Sign-On (SSO) solutions, leveraging authentication and authorization protocols including Security Assertion Markup Language (SAML) [27], OAuth2 [28], and OpenID Connect (OIDC) [29]. Through these integrations, it dynamically retrieves and refreshes tokens to enable seamless, compliant, and auditable access to resources.

In this experimental implementation, however, the Credential Manager replaces dynamic token retrieval with direct storage and lookup of static credentials, such as pre-generated service account keys or access keys, to simplify deployment and facilitate reproducible evaluation of permission-aware retrieval workflows. For example, GCP uses pre-generated service account keys, AWS uses access key ID and secret access key pairs, and Keycloak-based local services store client credentials in a static form. While this approach does not reflect production-scale orchestration, it simplifies the credential retrieval logic for evaluating the core permission validation functionality of the framework.

To optimize performance and reduce redundant authorization requests, the framework employs an internal caching mechanism for credentials with a time-to-live (TTL) configuration. This TTL-based caching ensures that credentials are reused within their validity window, while expired tokens are automatically refreshed to enforce security policies such as key rotation requirements [30].

In summary, while the experimental Credential Manager uses static credentials for simplicity, its modular design anticipates integration with dynamic, secure, and compliant credential orchestration mechanisms in production environments. To further clarify this production-grade integration,

we detail the complete dynamic credential orchestration workflow, which integrates with enterprise SSO systems, in Appendix B.

3) IAM ADAPTER

The IAM Adapter module is responsible for validating user access permissions for each retrieved document. It leverages platform-specific SDKs to communicate with the native IAM service of the document's provider (e.g., GCP, AWS, or Keycloak). To prevent access validation from becoming a performance bottleneck, the adapter employs a multithreading-based architecture to execute these I/O-bound permission checks in parallel. The adapter's modular design and specific implementation details are further described in Appendix C. Overall, the IAM Adapter provides a robust and extensible solution for validating user access permissions across heterogeneous services.

IV. EXPERIMENTAL SETUP

A. EXPERIMENTAL OVERVIEW

The primary objective of this experimental evaluation is to assess whether the proposed Permission-Aware RAG framework can accurately and effectively enforce document-level access control in realistic, heterogeneous multi-resource environments without requiring centralized policy merging. To achieve this, three distinct experiments were designed: (1) a scenario-based qualitative evaluation to analyze the correct application of access control rules in realistic user-policy-resource combinations across GCP, AWS, and Keycloak; (2) a large-scale quantitative evaluation to measure permission filtering scalability and its resulting impact on answer quality; and (3) a latency evaluation to measure the performance overhead of real-time permission validation by varying retrieval volume and concurrency.

All experiments were conducted on a server equipped with an Intel Xeon Gold 6230 CPU, 754GB of RAM, and three NVIDIA Tesla V100 GPUs. The experiments were conducted using the HotpotQA dataset, selected for its multi-hop questions that require retrieving information from multiple documents, simulating complex enterprise environments. Document contexts were encoded using the `all-MiniLM-L6-v2` Sentence Transformer model into 384-dimensional embeddings and stored in a centralized FAISS-based vector store. For final answer generation, the `gpt-4o-mini` model served as the LLM backend. To simulate a multi-cloud environment and ensure consistent network conditions, cloud resources were hosted in GCP's `asia-northeast3` and AWS's `ap-northeast-2` regions.

B. DATASET PREPARATION

For this study, we used the HotpotQA dataset [31] as the primary data source, given its wide adoption for evaluating multi-hop question answering systems. Specifically, we utilized the development set (distractor setting), which contains 13,783 unique supporting fact documents corresponding

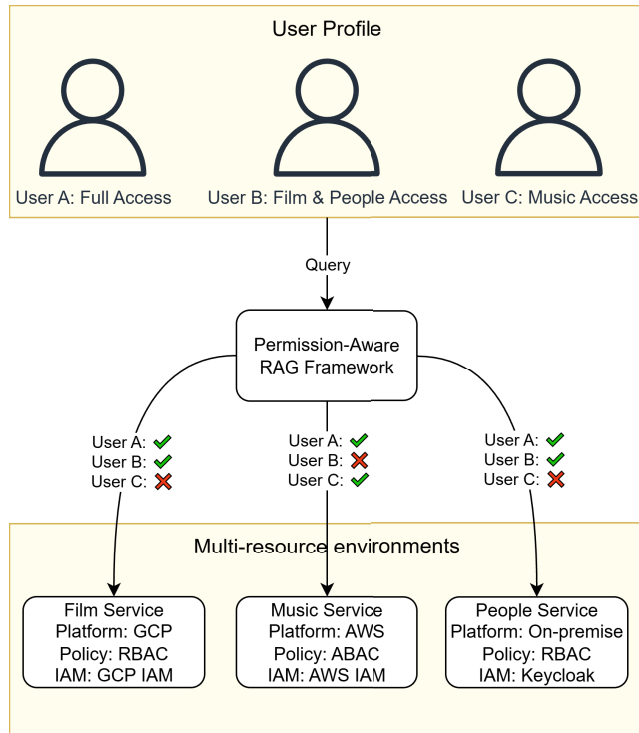


FIGURE 4. Setup for the scenario-based qualitative evaluation.

to 7,405 multi-hop questions, as the core corpus for our experiments. HotpotQA consists of Wikipedia-based articles containing multi-hop questions that require retrieving and reasoning over multiple documents, making it suitable for evaluating document-level retrieval and permission filtering in the proposed framework. In particular, since each question requires integrating information distributed across different documents, it effectively simulates realistic enterprise environments where resources are distributed across multiple services, the central RAG retrieves them, and each service's IAM system is used for real-time permission validation before integration. This characteristic provides a strong basis for evaluating the framework's ability to handle document-level retrieval and access filtering in complex multi-service settings.

1) DATA SELECTION AND FILTERING

We selected a subset of HotpotQA focusing on three main domains: films, music, and people. Questions and their associated supporting facts were filtered to ensure coverage across these domains. Here, *supporting facts* refer to the core evidence sentences within the original documents that are essential for inferring the answer to each question. The selection also aimed to simulate realistic multi-service enterprise scenarios, where different document types are managed by distinct services with separate IAM systems (e.g., film data in GCP, music data in AWS, and people data in Keycloak).

2) PREPROCESSING AND EMBEDDING UNIT CONFIGURATION

The original contexts in the HotpotQA dataset are structured as a list of self-contained sentences or short paragraphs for each document title. Recognizing that these are already meaningful units of information, this study utilized them directly as individual embedding units. Therefore, instead of applying an additional, arbitrary chunking process, we leveraged the dataset's inherent structure. This approach avoids the risk of semantic loss from artificial splitting and preserves the original context of each sentence.

C. QUALITATIVE EVALUATION: SCENARIO-BASED SETUP

A scenario-based qualitative evaluation verifies the framework's fine-grained, document-level enforcement under realistic, enterprise-like multi-service conditions and contrasts its behavior across heterogeneous services with a centralized-policy control group, thereby highlighting the robustness and advantages of the delegated IAM approach. Fig. 4 illustrates the scenario-based qualitative evaluation setup.

1) ENVIRONMENT CONFIGURATION

Three distinct services were configured to simulate a heterogeneous organizational environment:

- **Film Service (GCP IAM / RBAC):** Film documents governed by RBAC roles (e.g., *Film Producer* / full, *Film Intern* / partial)
- **Music Service (AWS IAM / ABAC):** Music documents controlled by ABAC using user attributes such as region and tier.
- **People Service (Keycloak IAM / RBAC):** People documents managed on an on-premise Keycloak server with RBAC roles (e.g., *Talent Agent*, *NoAccess*).

For the qualitative evaluation, a scenario-based subset of 20 multi-hop questions was sampled from HotpotQA. Each question requires at least two supporting documents, and the set spans all three domains (Film, Music, People). To reflect realistic enterprise boundaries, Film documents were stored in GCP, Music documents in AWS, and People documents in an on-premise service managed by Keycloak. The complete list of questions is provided in Appendix D. Policy-filtering correctness across heterogeneous IAM settings was evaluated using this scenario configuration.

2) USER PROFILES

Table 1 summarizes thirteen user configurations. The *Admin* user has unrestricted access to all services and serves as the upper bound for permission coverage and filtering correctness. Single-service users (F1, F2 for Film, M1, M2, M3 for Music, P1, P2 for People) represent typical enterprise roles or attributes confined to a single domain. Multi-service users (FM, FP, MP) combine heterogeneous IAM models across two domains, allowing us to verify that cross-service policy enforcement remains consistent.

TABLE 1. User access control settings across services, with specific role names and attributes to clarify permission scopes.

User ID	Service	Policy	Granted Permissions
Admin	All	Baseline	full access (all services)
F1	F	RBAC	Film Producer
F2	F	RBAC	Film Intern (partial access)
M1	M	ABAC	country=US, tier=premium
M2	M	ABAC	country=US, tier=basic
M3	M	ABAC	country=EU, tier=premium
P1	P	RBAC	Talent Agent (allow access)
P2	P	RBAC	NoAccess (deny access)
FM	F+M	RBAC, ABAC	F: Film Producer M: country=US, tier=premium
FP	F+P	RBAC, RBAC	F: Film Producer P: Talent Agent
MP	M+P	ABAC, RBAC	M: country=US, tier=premium P: Talent Agent
CG	F+M	Merged RBAC	Media Reviewer (Film Producer + tier=premium)

F: Film service, M: Music service, P: People service.

CG: Control Group with merged RBAC (country=US omitted).

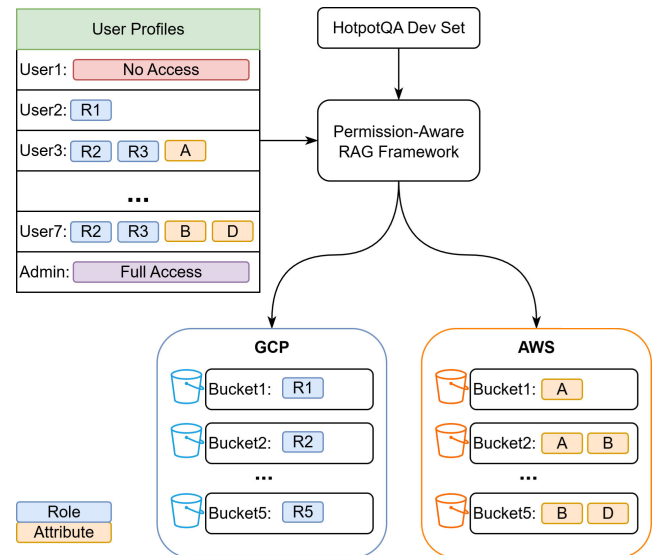
Control Group (CG) models a policy-merging scenario that would arise in the absence of our framework. An administrator consolidates heterogeneous rules into a single RBAC role named *Media Reviewer*, intending to replicate the FM user's privileges. Those privileges combine the Film service's *Film Producer* role with the Music service's ABAC conditions `country=US` and `tier=premium`.

During the manual process of consolidating heterogeneous policies into a single RBAC role, the `country=US` attribute is inadvertently omitted, leaving only `tier=premium`. As a result, the *Media Reviewer* role grants access to all premium-tier Music documents regardless of country, expanding its scope beyond the intended FM policy and creating a cross-region data-leakage risk.

CG therefore exposes the vulnerability of manual policy merging and serves as the control group for comparison with our distributed verification framework.

3) EVALUATION PROCEDURE

To qualitatively assess the accuracy of document-level access filtering, each of the 13 user profiles was given a set of 20 multi-hop questions. For each query, the framework retrieved the top-5 relevant documents from the centralized vector store, then validated each document against the provider's IAM (GCP, AWS, Keycloak). Using the expected allow/deny decision from the user's assigned policy as the ground truth, we compared it with the actual filtering result

**FIGURE 5.** Setup for the large-scale quantitative evaluation.

to verify whether unauthorized documents were consistently excluded and authorized ones correctly passed. We refer to this agreement as *Match*, which is formally defined in (2).

D. QUANTITATIVE EVALUATION: LARGE-SCALE SETUP

This large-scale quantitative evaluation was designed to verify two key objectives for the proposed Permission-Aware RAG framework. First, it confirms that the framework operates stably and accurately enforces document-level access control according to heterogeneous IAM policies in a large-scale, multi-cloud environment that simulates real-world operating conditions. This is essential for demonstrating the system's security robustness and operational reliability. Second, it quantitatively analyzes the impact of strict access control on final answer quality. To this end, we measure the correlation between a user's permission scope (Permission Coverage) and answer accuracy (EM and F1 Score) to clearly define the inherent trade-off between security and answer quality. An overview of the experimental environment is shown in Fig. 5.

1) USER ENVIRONMENT AND DOCUMENT DISTRIBUTION

For our experiments, we used the 13,783 unique supporting fact documents, corresponding to 7,405 questions, from the HotpotQA development set (distractor setting). These documents were evenly distributed across Google Cloud Platform (GCP), which enforced Role-Based Access Control (RBAC) policies, and Amazon Web Services (AWS), which enforced Attribute-Based Access Control (ABAC) policies. Within each environment, we configured five distinct buckets and randomly assigned the documents to them. In the GCP environment, each bucket was bound to a unique IAM role (Role 1–5), while in the AWS environment, each bucket was associated with specific combinations of predefined attributes (e.g., Attribute A, B, C).

TABLE 2. Summary of experimental evaluations and key findings.

Evaluation	Objective	Key Finding
Qualitative (Scenario-based)	Verify fine-grained policy enforcement and compare against a 'policy merging' baseline.	The framework correctly enforces heterogeneous policies, in contrast to a flawed 'policy merging' approach (used as a baseline) which is shown to cause data leakage.
Quantitative (Large-scale)	Assess policy enforcement accuracy at scale and quantify the trade-off between access rights and QA performance.	Achieved perfect policy enforcement ('Match: PASS'). A clear positive correlation exists between Permission Coverage and EM/F1 scores.
Latency	Measure the performance overhead and stability of the real-time permission validation step.	The parallelized architecture's low performance overhead demonstrates its feasibility for real-time, secure applications.

Detailed results for each evaluation are provided in the subsequent sections.

TABLE 3. RBAC access control results for representative queries.

Q#	Filename [svc]	A	F1	F2	Match
Q1	Home on the Range [F]	O	O	X	PASS
	Pete's Dragon [F]	O	O	X	PASS
	Song of the South [M]	O	X	X	PASS
Q2	Crazy English [F]	O	O	O	PASS
	The River [F]	O	O	O	PASS
	Bridge over [M]	O	X	X	PASS
	Amy [F]	O	O	X	PASS
	Oasis: Supersonic [M]	O	X	X	PASS

O: Permitted, X: Denied, svc: Service Type, A: Admin.

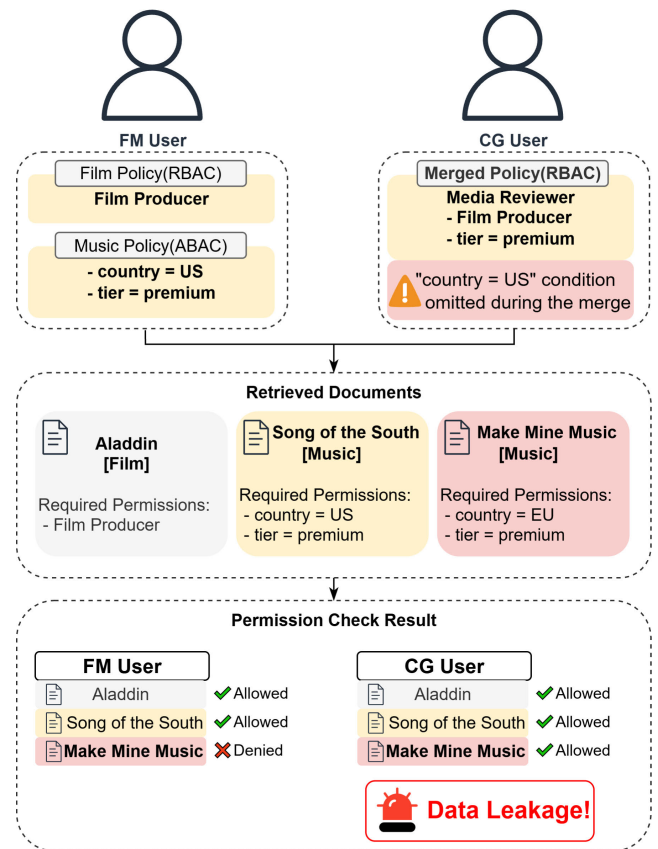
Bridge over: Bridge over Troubled Water.

TABLE 4. Cross-service access control results.

Q#	Filename [svc]	CG	FM	FP	MP	Match
Q9	Song of the South [M]	O	O	X	O	PASS
	Aladdin [F]	O	O	O	X	PASS
	Aladdin [F]	O	O	O	X	PASS
	Make Mine Music [M]	O	X	X	O	PASS
	Make Mine Music [M]	O	X	X	O	PASS
Q14	Last of the Duanes [F]	O	O	O	X	PASS
	George Montgomery [P]	X	X	O	O	PASS
	Last of the Duanes [F]	O	O	O	X	PASS
	Last of the Duanes [F]	O	O	O	X	PASS
	Pete's Dragon [F]	O	O	O	X	PASS
Q20	Loch Lomond [M]	O	O	X	O	PASS
	John Kirby [P]	X	X	O	O	PASS
	John Kirby [P]	X	X	O	O	PASS
	Bridge over [M]	O	O	X	O	PASS
	Once [M]	O	X	X	O	PASS

Loch Lomond: The Bonnie Banks o' Loch Lomond.

We also defined eight user profiles with varying levels of access to simulate diverse permission scopes, including a baseline admin user with full access. The remaining users were configured with scoped roles and attribute combinations to reflect restricted access patterns. The detailed

**FIGURE 6.** Data leakage resulting from flawed policy merging.**TABLE 5.** Quantitative evaluation results across users.

User	PermCov(%)	Match	EM	F1	EM@Admin
user-1	0.00	PASS	0.24 ± 0.42	0.31 ± 0.43	0.41
user-2	9.48	PASS	0.23 ± 0.42	0.30 ± 0.43	0.41
user-3	29.98	PASS	0.28 ± 0.45	0.35 ± 0.45	0.52
user-4	41.16	PASS	0.28 ± 0.45	0.37 ± 0.45	0.54
user-5	50.38	PASS	0.34 ± 0.47	0.43 ± 0.46	0.67
user-6	60.72	PASS	0.34 ± 0.47	0.45 ± 0.46	0.66
user-7	69.84	PASS	0.38 ± 0.48	0.50 ± 0.46	0.75
admin	100.00	PASS	0.47 ± 0.50	0.61 ± 0.44	1.00

configurations for both the resource access requirements and the user profiles are provided in Appendix E.

TABLE 6. Combined Latency by Top-k and Worker Concurrency (ms). The first block varies Top-k with Workers fixed at 20; the second block varies Workers with Top-k fixed at 10. Values are reported as mean $\pm$ std.

Top-k	Workers	IAM Latency	E2E Latency	GCP Latency	AWS Latency
1	20	64.94 $\pm$ 54.34	669.37 $\pm$ 158.62	49.31 $\pm$ 5.12	74.28 $\pm$ 68.21
3	20	81.44 $\pm$ 52.88	700.92 $\pm$ 176.16	51.11 $\pm$ 8.24	46.24 $\pm$ 54.70
5	20	93.28 $\pm$ 55.34	868.77 $\pm$ 686.17	49.06 $\pm$ 5.74	43.46 $\pm$ 52.96
10	20	123.14 $\pm$ 53.25	784.49 $\pm$ 265.79	56.94 $\pm$ 11.64	38.80 $\pm$ 33.14
15	20	170.42 $\pm$ 63.28	872.56 $\pm$ 314.20	66.33 $\pm$ 18.93	45.85 $\pm$ 41.51
20	20	203.56 $\pm$ 60.97	840.36 $\pm$ 194.35	74.74 $\pm$ 28.19	46.88 $\pm$ 39.29
10	1	420.71 $\pm$ 298.50	1316.78 $\pm$ 853.72	51.88 $\pm$ 30.33	32.67 $\pm$ 57.67
10	2	230.67 $\pm$ 193.80	1019.58 $\pm$ 1075.89	50.12 $\pm$ 42.95	34.87 $\pm$ 53.65
10	4	148.86 $\pm$ 73.00	801.92 $\pm$ 337.96	52.22 $\pm$ 17.57	32.39 $\pm$ 41.29
10	8	141.17 $\pm$ 77.23	836.81 $\pm$ 366.77	58.37 $\pm$ 30.79	39.54 $\pm$ 39.64
10	10	184.16 $\pm$ 442.70	1012.74 $\pm$ 854.12	56.84 $\pm$ 9.58	54.40 $\pm$ 203.48
10	15	127.33 $\pm$ 51.15	842.39 $\pm$ 369.67	56.86 $\pm$ 8.22	39.78 $\pm$ 37.34
10	20	128.43 $\pm$ 63.05	742.28 $\pm$ 163.22	58.28 $\pm$ 20.39	39.67 $\pm$ 39.37

TABLE 7. Resource metadata schema fields managed by the data ingestion module.

Field	Example Value
resourceID	1dca6e08-f1d4-402b-...
local_resource_id	abc
provider	gcp
endpoint	https://storage.googleapis.com
file_name	report.pdf
parameters.region	asia-northeast3
parameters.bucket	jxxjndxu-gcp-bucket-5
parameters.object_key	folder1/report.pdf

2) EVALUATION PROCEDURE AND METRICS

With the GCP (RBAC) and AWS (ABAC) environments fully configured, we evaluated the system using 1,000 randomly sampled questions from the HotpotQA development set of 7,405 questions. For each query, the vector retriever selected semantically relevant documents using a fixed $\text{top-}k$ retrieval with $k = 5$. These documents were filtered through real-time IAM validation. Only the authorized subset was passed to the LLM for answer generation.

Permission Coverage (PermCov) quantifies the proportion of all retrieved documents that a user is actually authorized to access, as formally defined in (1):

$$\text{PermCov} = \frac{|D_{\text{auth}}|}{|D_{\text{retr}}|} \times 100 \quad (1)$$

where $|D_{\text{auth}}|$ is the number of retrieved documents (aggregated over all queries) the user is actually authorized to access, and $|D_{\text{retr}}|$ is the total number of retrieved documents across those queries (e.g., $N \times k$ for N queries with $\text{top-}k = k$).

This metric indicates how restrictive a user's effective permission scope is: lower values mean the user can view only a small portion of the retrieved set (i.e., most retrieved documents are inaccessible). In other words, PermCov measures how many unauthorized documents are mixed

into retrieval results and thus highlights the necessity of permission-aware document access in RAG systems.

Match indicates whether the system's IAM enforcement exactly agrees with the user's true permissions for all retrieved documents, which is formally defined in (2). This was verified programmatically using the following procedure:

- **Ground-Truth Generation:** For each user profile, we established a ground-truth by pre-generating a definitive list of all documents they were permitted to access.
- **Verification:** The set of documents the framework deemed accessible for a given query was compared against the ground-truth list. Any document incorrectly included in the results was a False Positive (FP), and any document from the ground-truth list that was incorrectly excluded was a False Negative (FN).

$$\text{Match} = \begin{cases} \text{PASS}, & \text{if } (\text{FP} + \text{FN}) = 0 \\ \text{FAIL}, & \text{otherwise} \end{cases} \quad (2)$$

where FP and FN denote false positives and false negatives in permission checks, respectively.

Following standard QA evaluation practice [31], we use Exact Match (EM) and F1 Score to evaluate the final answer quality [32]. EM checks if the model's predicted answer exactly matches the ground-truth, while F1 computes the token-level overlap.

In addition, we introduce a supplemental metric, EM@Admin, which measures the Exact Match score only over the subset of questions that the admin user answered correctly. This metric captures the performance degradation attributable solely to permission filtering, isolating the impact of restricted access from cases where the question is inherently unanswerable even with full access. By comparing EM and EM@Admin, we can assess how much access control affects the ability to answer questions that are, in principle, answerable.

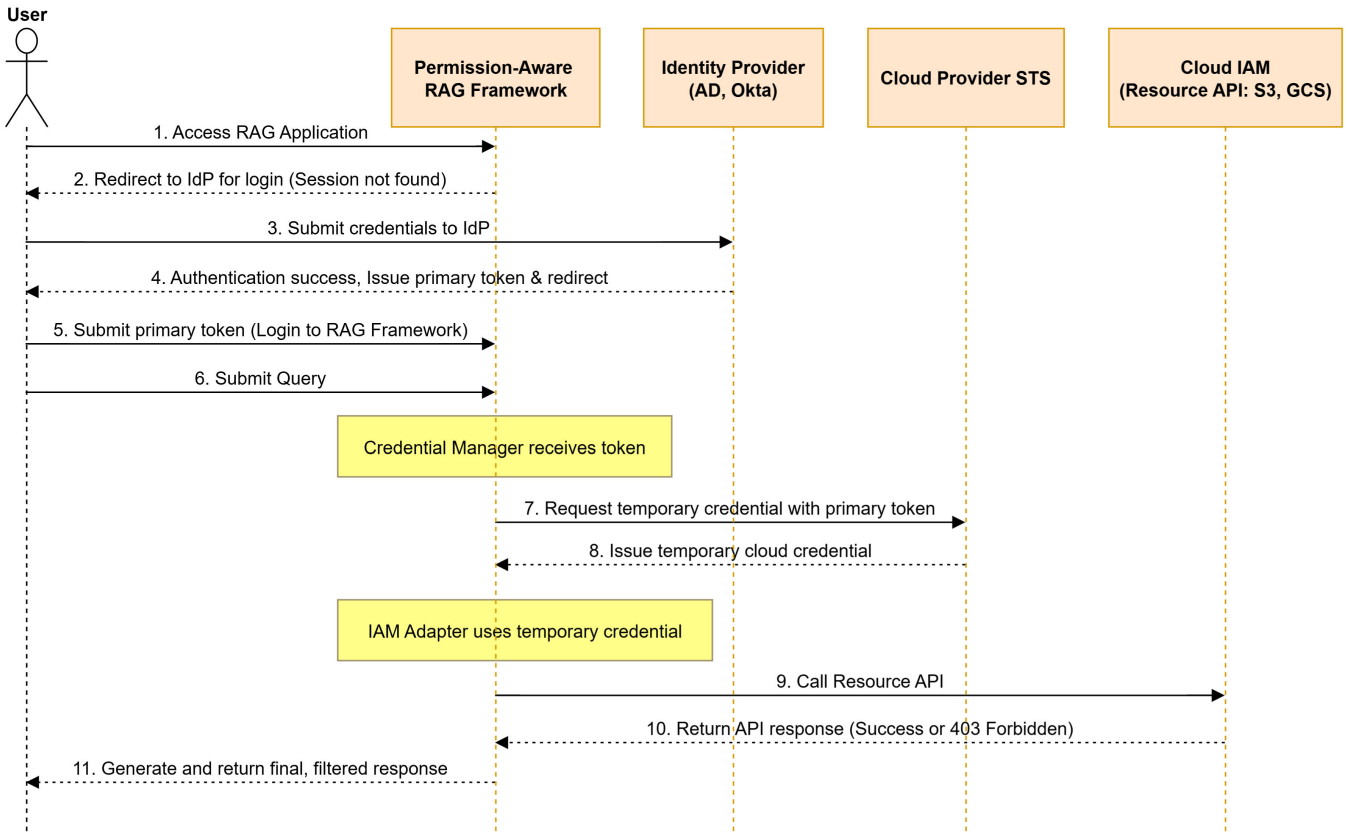


FIGURE 7. Sequence diagram of the dynamic credential orchestration workflow via enterprise SSO.

This evaluation verifies the system’s ability to precisely enforce document-level access control while maintaining answer quality. By comparing EM and F1 scores across users with different permission scopes, we analyze the trade-offs between secure filtering and QA accuracy.

E. LATENCY EVALUATION: IMPACT OF PERMISSION RETRIEVAL AND CONCURRENCY

This experiment was designed to evaluate the latency overhead introduced by real-time permission retrieval in the Permission-Aware RAG framework. In multi-cloud environments where each retrieved document requires validation against its native IAM system, permission checks can become a performance bottleneck. This evaluation aims to measure how such latency scales with increased number of retrieved documents and concurrency. To accurately measure the raw overhead of each real-time API call, this evaluation bypassed any caching of the permission check results (i.e., the allow/deny decisions). Such a mechanism, which would typically be managed with a Time-to-Live (TTL), was disabled to ensure every check was validated externally.

1) OBJECTIVE AND METRICS

The primary objective is to assess how the permission validation process affects system responsiveness. We focus on three latency dimensions:

- **IAM Latency:** Time taken to complete permission checks for all top- k retrieved documents via IAM APIs. Measured from the start of validation until all responses are returned.
- **End-to-End (E2E) Latency:** Total processing time from query submission to final LLM response.
- **GCP Latency:** Average API response time for permission checks from GCP IAM.
- **AWS Latency:** Average API response time for permission checks from AWS IAM.

2) EXPERIMENTAL DESIGN

To measure the system’s latency under varying permission retrieval loads, we conducted two sets of experiments using an *admin-level user* with full access to all documents. Both experiments were conducted under the same setup as the quantitative evaluation and were run on 50 queries sampled from the HotpotQA dev set.

Top- k Variation. We fixed the number of concurrent permission threads to 20 and varied the number of retrieved documents per query (top- k) from 1 to 20. This setup evaluates the cost of permission checking as more documents are validated.

Worker Concurrency Variation. We fixed top- k at 10 and varied the number of permission-checking threads (max-workers) from 1 to 20. This experiment measures the

TABLE 8. Scenario-based question samples for qualitative evaluation. Each question is paired with two supporting facts from distinct documents in different services ([F]=Film, [M]=Music, [P]=People).

Q#	Question	Supporting Fact 1	Supporting Fact 2
Q1	Which film was released first, Home on the Range or Pete's Dragon?	Pete's Dragon (2016 film) [F]	Home on the Range (2004 film) [F]
Q2	Was the documentary "The River" made for the same reason as "Crazy English"?	Crazy English (film) [F]	The River (1938 film) [F]
Q3	Who performed the Simon & Garfunkel song that was a follow-up single to "The Boxer" on Australian Idol?	Rain (Anthony Callea song) [M]	Bridge over Troubled Water (song) [M]
Q4	Who collaborated with Rihanna on the song "Numb"?	The Monster (song) [M]	Numb (Rihanna song) [M]
Q5	Who is older, Annie Morton or Terry Richardson?	Annie Morton (model) [P]	Terry Richardson (photographer) [P]
Q6	What year did Guns N' Roses perform a promo for a movie starring Arnold Schwarzenegger as a former NYPD detective?	End of Days (film) [F]	Oh My God (Guns N' Roses song) [M]
Q7	Which film was released more recently, Make Mine Music or Old Dogs?	Make Mine Music (1946 film) [F]	Old Dogs (film) [F]
Q8	What year did the musical based on the 2007 Irish film of the same name premiere?	Once (musical) [M]	Once (film) [F]
Q9	Are Aladdin and Song of the South produced by the same company?	Aladdin (2019 film) [F]	Song of the South (film) [F]
Q10	Which producer from the 2015 Amy Winehouse documentary also worked on Oasis: Supersonic?	Oasis: Supersonic [M]	Amy (2015 film) [F]
Q11	"The Look of Love" appeared in Casino Royale (1967). That film was based on a novel by which author?	Casino Royale (1967 film) [F]	The Look of Love (1967 song) [M]
Q12	Which Beatles movie did the John's Children album Orgasm dub audience screams from?	Orgasm (John's Children album) [M]	A Hard Day's Night (film) [F]
Q13	The song "Medicine" by The 1975 appears in a film by which Danish director?	Drive (2011 film) [F]	Medicine (The 1975 song) [M]
Q14	Who directed the 1941 film based on Zane Grey's novel, starring an actor who was also an artist and stuntman?	George Montgomery (actor) [P]	Last of the Duanes (1941 film) [F]
Q15	Who wrote the 1996 historical drama film in which William Preston appeared?	William Preston (actor) [P]	The Crucible (1996 film) [F]
Q16	Which rapper worked with a star of 8 Mile and also worked with Anita Baker's keyboardist?	8 Mile (film) [F]	Luis Resto (musician) [P]
Q17	Where was the singer of "B Boy" raised?	B Boy (song) [M]	Meek Mill [P]
Q18	In which year did the artist who wrote "Hyperreal" release his debut studio album?	Hyperreal (song) [M]	Flume (musician) [P]
Q19	Jim Price appeared on the Rolling Stones album released in what month?	Jim Price (musician) [P]	Goats Head Soup [M]
Q20	Which song scored by John Kirby is often played at the end of an evening of revelry?	John Kirby (musician) [P]	The Bonnie Banks o' Loch Lomond [M]

TABLE 9. Resource and access control requirements.

Environment	Bucket ID	Required Permission
GCP (RBAC)	...-gcp-bucket-1	Role 1
	...-gcp-bucket-2	Role 2
	...-gcp-bucket-3	Role 3
	...-gcp-bucket-4	Role 4
	...-gcp-bucket-5	Role 5
AWS (ABAC)	...-aws-bucket-1	Attribute A
	...-aws-bucket-2	Attribute A and Attribute B
	...-aws-bucket-3	Attribute C
	...-aws-bucket-4	Attribute B and Attribute C
	...-aws-bucket-5	Attribute B

scalability of parallel permission validation and its impact on latency.

V. EVALUATION

This section presents a comprehensive evaluation of the Permission-Aware RAG framework through three distinct experiments: a qualitative analysis, a large-scale quantitative

TABLE 10. User profiles and assigned permissions.

User ID	GCP Roles	AWS Attributes
user-1	-	-
user-2	Role 2	-
user-3	Role 2, Role 3	Attribute C
user-4	Role 1, 3, 5	Attribute A
user-5	Role 1, 2, 3	Attribute B, Attribute C
user-6	Role 1, 2, 3, 4	Attribute A, Attribute C
user-7	Role 1 - 5	Attribute A, B, C
baseline-admin	All Roles	All Attributes

analysis, and a latency evaluation. Table 2 provides a high-level summary of each evaluation's objective and key findings.

A. QUALITATIVE EVALUATION: A SCENARIO-BASED ANALYSIS

The qualitative evaluation began by validating the framework's baseline accuracy and core design principles in a single-service RBAC scenario. The results in Table 3

TABLE 11. Illustrative summary of edge cases and candidate mitigation strategies.

Edge Case	Description	Impact / Problem	Mitigation Strategy
IAM Service Outage	The IAM system (e.g., GCP, AWS, Keycloak) fails to respond or returns an error.	Permission validation becomes impossible, leading to system performance degradation due to timeouts.	Consider IAM health checks with a fallback flag to trigger a predefined behavior (e.g., deny all access or use cached permissions).
No Permissions in Top-k Results	The user lacks permission for all documents retrieved by the vector search, resulting in an empty context.	The LLM may hallucinate or generate an answer without a factual basis.	When no context is available, switch to a prompt optimized for leveraging the LLM's self-knowledge and clearly state that the answer is not based on retrieved documents.
Inconsistent Metadata	Metadata required for permission checks (e.g., resource location, IAM info) is missing or out of sync.	Failure in permission validation, potentially leading to incorrect access decisions (denial of valid access or failure to check permissions).	Implement a periodic, automated process to synchronize and verify consistency between the metadata store and the source IAM systems.
Permission Change Latency	When permission results are cached for performance, a user's recent permission change might not be reflected immediately.	Temporary incorrect access (either denial of new access or prolonged access after revocation).	Apply a Time-To-Live (TTL) to all cached permission results. Implement a mechanism to forcibly invalidate the cache for a user or resource immediately after a known permission change.

demonstrated both *inter-service segregation*, as the F1 user (Film Producer) was correctly blocked from non-Film documents, and *intra-service granularity*, as the F2 user (Film Intern) was granted access to only a specific subset of Film documents. The consistent PASS scores confirmed the reliability of the document-level enforcement mechanism, providing a solid foundation for more complex evaluations.

The evaluation next compared our delegated approach against a **manual policy merging** baseline in a cross-service scenario, revealing a critical security vulnerability. As detailed in Table 4, the CG (Control Group) user, operating under a flawed merged policy, was granted access to the “Make Mine Music” document where the FM user was correctly denied. This failure, caused by an omitted attribute during the manual policy translation, is visualized in Fig. 6. The figure clearly illustrates the security risks introduced by manual policy merging compared to the precise, policy-compliant enforcement of our framework.

This outcome highlights that manual policy consolidation is not only error-prone but also faces fundamental architectural limitations. Attempting to translate complex ABAC policies into RBAC, for instance, can lead to a “role explosion” that renders the system difficult to manage. In contrast, our framework's delegated approach circumvents these challenges entirely by enforcing the original policies through their native IAM systems. The qualitative evaluation thus confirms our method as a more practical and robust solution for maintaining security and policy integrity in complex, multi-resource environments.

B. QUANTITATIVE EVALUATION: LARGE-SCALE ANALYSIS

To assess the scalability and policy enforcement accuracy of the proposed Permission-Aware RAG framework under realistic enterprise conditions, we conducted a large-scale evaluation using 1,000 multi-hop questions and

approximately 13,783 documents distributed across GCP and AWS. Eight user profiles with varying access levels were designed to simulate fine-grained permission settings.

Table 5 presents the results across all users. All profiles achieved a perfect *Match* score, indicating that the framework successfully enforced real-time, document-level access control for every retrieved document in accordance with each user's IAM policy, even at scale.

Moreover, we observe a clear positive correlation between Permission Coverage (PermCov) and QA performance. As users gain access to a greater portion of the document corpus, their EM and F1 scores improve accordingly. For instance, user-2 (PermCov 9.48%) achieved an F1 of 0.30, while user-7 (69.84%) scored 0.50. Furthermore, the standard deviations for both EM and F1 scores are relatively consistent across all user profiles, indicating that the framework's QA performance is stable regardless of the user's access level. This trade-off quantifies the cost of limited access in terms of answer quality, offering practical insight for balancing security and utility.

To isolate the impact of access restrictions from question difficulty, we introduce **EM@Admin**, a metric that computes EM only over the subset of questions correctly answered by the admin. As shown in Table 5, the EM@Admin score consistently exceeds the standard EM score for all non-admin users. This clearly indicates that restricted document access directly reduces the availability of crucial supporting documents necessary for correctly answering the questions. The EM@Admin metric thus isolates the impact of access restrictions from inherent question difficulty, highlighting the tangible trade-off between fine-grained permission control and achievable answer accuracy. Users with broader access consistently achieve higher EM@Admin scores. For example, user-3 reached 0.52, while user-7 achieved 0.75. This confirms that performance differences are

attributable to access constraints rather than inherent question complexity.

Interestingly, user-1, who had no access to any documents (PermCov 0.00%), still achieved a non-trivial F1 score of 0.31. This suggests that the LLM may have relied on its internal knowledge or correctly guessed answers to binary (e.g., Yes/No) questions. In cases where no documents are retrieved, applying adaptive prompting, as demonstrated in FIT-RAG [33], can help the model better utilize its self-knowledge and improve response quality [34].

Overall, these results validate that the Permission-Aware RAG framework robustly enforces heterogeneous access policies while preserving predictable QA performance. It enables secure, explainable, and operationally reliable deployment in enterprise environments requiring fine-grained access control.

C. LATENCY EVALUATION: IMPACT OF PERMISSION RETRIEVAL AND CONCURRENCY

This section analyzes the latency evaluation results, focusing not just on performance metrics but on their practical implications for real-world deployment. The experiments confirm that the framework's architecture effectively balances real-time security with high performance. To accurately measure the raw overhead of each real-time API call, the caching of permission check results was intentionally disabled for this evaluation.

1) LATENCY BY NUMBER OF RETRIEVED DOCUMENTS

As shown in the top block of Table 6, when the number of retrieved documents (*top-k*) was increased from 1 to 20, the corresponding IAM Latency for permission checks increased modestly by about 140 ms. In sharp contrast, the total End-to-End (E2E) Latency was substantially higher. Furthermore, the standard deviation data reveals a key insight into performance stability: while the IAM Latency shows a relatively low and stable standard deviation, the E2E Latency exhibits significantly higher volatility.

These results confirm that the latency introduced by permission validation is negligible compared to the inference time of the LLM, indicating that access control does not impose a performance bottleneck. The primary factor influencing user-perceived latency is the LLM, not the security check. This alleviates the major concern of performance degradation for enterprise adoption and confirms the framework's ability to scale efficiently with increasing data loads.

2) LATENCY BY WORKER CONCURRENCY

The bottom block of Table 6 illustrates the effect of parallelism. Increasing the worker count from 1 to 15 reduces the mean IAM Latency by approximately 70% in total (from 420.71 ms to 127.33 ms), with the most substantial reduction occurring between 1 and 4 workers. However, beyond 4 workers, the performance gains diminish, indicating

that the system is approaching a state of saturation. Although some conditions show high standard deviation due to external network factors, the standard deviation of IAM Latency remains consistently and substantially smaller than that of the total E2E Latency. This confirms that even when accounting for real-world network volatility, the parallel permission validation step is not the primary driver of unpredictability in the system. This provides a data-driven guideline for tuning an optimal number of workers to balance speed and stability in a real-world deployment.

The findings empirically demonstrate that integrating IAM-based, fine-grained access control into high-throughput RAG systems does not lead to critical performance degradation, thereby supporting the feasibility of secure deployment in enterprise environments.

VI. CONCLUSION AND FUTURE WORK

This paper introduced the Permission-Aware RAG framework, a novel architecture for enforcing real-time, document-level access control in retrieval-augmented generation systems operating across heterogeneous, multi-resource environments. By delegating access validation to native IAM systems, the framework preserves policy autonomy and eliminates the need for centralized policy merging, which is often error-prone and operationally burdensome.

The experimental evaluations collectively demonstrate that the framework reliably enforces fine-grained permissions, scales effectively across diverse services, and maintains stable and predictable low-latency performance suitable for real-time applications. These findings confirm the framework's ability to support secure, policy-compliant retrieval without compromising responsiveness or answer quality. Its plug-and-play design allows seamless integration with existing IAM infrastructures such as RBAC and ABAC, enabling flexible RAG deployment across independently governed services.

In contextualizing the advantages of our design, we compare it against alternative approaches. First, *manual policy merging*, as demonstrated by our qualitative evaluation baseline (the CG user), is error-prone and can introduce critical security risks like data leakage. This approach is not only operationally burdensome but often leads to a loss of policy granularity during translation. Second, *static filtering*, which involves pre-building separate indexes for each user role or group, is fast but highly inflexible. It requires index regeneration upon permission changes, causes unnecessary data duplication when the same document is shared across multiple groups, and inevitably suffers from stale permission issues in dynamic environments. In contrast, *index-level controls* filter based on metadata within a single index, which can emulate RBAC and simple ABAC. However, this approach requires continuous metadata synchronization for complex or dynamic policies and struggles to reflect time-based conditions or hierarchical resource policies in real-time. Moreover, query costs can increase with the number of filter conditions, leading to performance degradation

in large-scale environments. Our framework's approach of *delegating validation to native IAMs in real-time* directly overcomes these limitations. It eliminates the risks of manual policy mapping, ensures that the most current permissions are always enforced, and leverages the full expressiveness of IAM, thereby providing superior security, dynamism, and compliance in complex enterprise environments.

Despite these strengths, several practical challenges and limitations remain. The framework's reliability is influenced by the availability and responsiveness of external IAM services, making it vulnerable to issues like IAM outages. Furthermore, while the dataset utilized in this study was well-suited for evaluating the core mechanisms of multi-hop question answering and permission-aware retrieval, it does not fully capture the scale, heterogeneity, and policy complexity of proprietary enterprise environments. Consequently, the results should be interpreted with this limitation in mind, and future work should extend the evaluation to more diverse datasets and realistic IAM configurations.

To address these limitations, future work will focus on enhancing the framework's robustness for production environments. We have identified several edge cases, such as those mentioned above, and a detailed breakdown of these cases with candidate mitigation strategies is outlined in Appendix F as preliminary considerations. Key areas of future research will include implementing robust TTL-based caching to handle API rate limits and reduce latency, and introducing health checks with fallback mechanisms to improve fault tolerance. Additionally, we plan to explore new retrieval strategies to enhance answer quality in low permission coverage scenarios. Finally, to further enhance security, we aim to encrypt both document embeddings and metadata within the vector store.

In summary, the Permission-Aware RAG framework establishes a secure, scalable, and adaptable foundation for building trustworthy, access-controlled AI systems in real-world, multi-tenant environments. The implementation of the proposed framework is publicly available as open-source code on GitHub [35].

APPENDIX A RESOURCE METADATA SCHEMA

Table 7 details the metadata schema managed by the Data Ingestion module.

APPENDIX B PRODUCTION-GRADE CREDENTIAL ORCHESTRATION WORKFLOW

This appendix details the practical workflow for the Credential Manager in a production environment. It illustrates how the framework implements **Identity Federation** by integrating enterprise Single Sign-On (SSO) systems with cloud Security Token Services (STS). The process, illustrated in Fig. 7, adheres to the principle of least privilege by avoiding the use of static, long-lived keys.

The workflow consists of the following key stages:

- **User Authentication via SSO (Steps 1-5)**

When a user first accesses the RAG application, the

RAG framework redirects them to a corporate Identity Provider (IdP) like Okta or Azure AD for authentication. Upon successful authentication, the IdP issues a primary identity token (e.g., an OIDC ID Token or a SAML Assertion), which is returned to the RAG framework. The framework validates this token to establish a secure, authenticated session for the user. This initial step handles user authentication securely without the RAG system ever handling user passwords.

- **Dynamic & Temporary Credential Issuance (Steps 6-8)**

Once authenticated, the user submits a query to the RAG framework (Step 6). The **Credential Manager** then uses the user's identity information from the established session to request a temporary, short-lived credential from the cloud provider's Security Token Service (STS) (Step 7). The STS, in turn, issues the temporary credential back to the framework (Step 8). This "token exchange" process is central to the framework's security, as it ensures that permissions are granted dynamically and expire automatically.

- **Permission-Aware Resource Access (Steps 9-11)**

The **IAM Adapter** uses the newly issued temporary credential to make an authorized API call to the native cloud service. The cloud provider's IAM service validates the credential and the requested permission, returning a success or failure response (Step 10). Based on these real-time checks, the RAG framework processes only the authorized documents to generate and return the final, filtered response to the user (Step 11).

This end-to-end workflow demonstrates how the Permission-Aware RAG framework can be securely deployed in complex enterprise environments by integrating with existing identity management systems and enforcing dynamic, just-in-time access control.

APPENDIX C IAM ADAPTER IMPLEMENTATION DETAILS

The IAM Adapter is designed with a modular and extensible architecture to validate user access permissions across heterogeneous platforms. Each implementation class extends a shared interface defined in the abstract base class `AccessControlBase`, which declares the common `check_permission()` method. Service-specific classes, including `GCPAccessControl`, `AWSAccessControl`, and `KeycloakAccessControl`, inherit from this base class and implement the method using their respective native IAM APIs and SDKs. Specifically, the `boto3` library is used for AWS, the `google-cloud-storage` SDK for GCP, and REST API endpoints for Keycloak-based services.

To enhance system scalability and response performance, the IAM Adapter employs a multithreading-based architecture. Permission checks are predominantly I/O-bound due to network API calls, and executing them in parallel prevents access validation from becoming a bottleneck in the retrieval pipeline. The current implementation leverages Python's

`ThreadPoolExecutor` class from the concurrent library to manage thread pools and schedule tasks concurrently, ensuring efficient resource utilization.

Furthermore, the adapter's modular design is intentionally extensible to support more advanced parallel processing schemes. For instance, in large-scale production environments, the adapter can be readily integrated with multiprocessing architectures that leverage distributed task queues such as Celery or RabbitMQ. These systems enable the distribution of permission-checking tasks across multiple worker nodes, thereby further increasing throughput and reliability.

Overall, the IAM Adapter provides a robust and extensible solution for validating user access permissions across heterogeneous services. By leveraging platform-specific SDKs, parallel execution, and support for advanced distributed processing, it ensures both secure and efficient operation within the Permission-Aware RAG framework.

APPENDIX D SCENARIO-BASED EVALUATION QUESTIONS

This appendix provides the full list of scenario-based evaluation questions used in qualitative evaluation. See Table 8.

APPENDIX E USER AND RESOURCE CONFIGURATION FOR QUANTITATIVE EVALUATION

This appendix details the user profiles and resource configurations created for the large-scale quantitative evaluation (Section IV-D). This setup was designed to create a transparent and reproducible testbed with heterogeneous IAM policies.

First, to establish a clear and reproducible benchmark, we defined specific access control requirements for the resources used in our evaluation. These resources are distributed across two distinct policy environments. The first is Google Cloud Platform (GCP), which utilizes a Role-Based Access Control (RBAC) model, and the second is Amazon Web Services (AWS), which employs an Attribute-Based Access Control (ABAC) model. Table 9 provides a detailed breakdown of these requirements, specifying the exact IAM role needed to access each GCP bucket and the particular combination of user attributes required for each AWS bucket.

Next, Table 10 shows the eight user profiles configured for the evaluation. These profiles were intentionally designed to test the framework's performance under varying levels of data accessibility, from a user with no permissions ('user-1') to one with complete access ('baseline-admin'). Each profile is assigned a unique combination of GCP's role-based permissions (RBAC) and AWS's attribute-based permissions (ABAC), allowing us to evaluate the system's ability to handle heterogeneous policies concurrently. The 'baseline-admin' user, who has unrestricted access, serves as the upper bound for measuring maximum permission coverage and achievable answer quality. The detailed settings for each profile are

also available in the publicly accessible source code for reproducibility.

APPENDIX F EDGE CASES AND MITIGATION STRATEGIES

Deploying the proposed framework in a production setting requires careful consideration of potential edge cases that could impact its stability and security. To outline preliminary directions, we have identified several scenarios, ranging from external service failures to specific permission-handling situations. Table 11 provides a summary of these edge cases and possible mitigation strategies.

REFERENCES

- [1] T. B. Brown, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2022, pp. 1877–1901.
- [2] E. G. Woo, M. C. Burkhart, E. Alsentzer, and B. K. Beaulieu-Jones, "Synthetic data distillation enables the extraction of clinical information at scale," *npj Digit. Med.*, vol. 8, no. 1, pp. 1–13, May 2025.
- [3] Y. Chen and P. Esmailzadeh, "Generative AI in medical practice: In-depth exploration of privacy and security challenges," *J. Med. Internet Res.*, vol. 26, Mar. 2024, Art. no. e53008, doi: [10.2196/53008](https://doi.org/10.2196/53008).
- [4] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," *J. Legal Anal.*, vol. 16, no. 1, pp. 64–93, Jan. 2024.
- [5] X. Li, Z. Li, S. Chen, Y. Xu, Q. Du, M. Tan, J. Huang, and W. Lin, "AlphaFin: Benchmarking financial analysis with retrieval-augmented stock-chain framework," in *Proc. LREC-COLING*, May 2024, pp. 773–783.
- [6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [8] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 6491–6501.
- [9] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," Nat. Inst. Stand. Technol., Gaithersburg, MD, USA, Tech. Rep. SP 800-207, Aug. 2020.
- [10] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman, "Role-based access control models," *IEEE Comput.*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [11] V. C. Hu, D. F. Ferraiolo, and D. R. Kuhn, *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, document NIST Special Publication 800-162, Jan. 2014.
- [12] Y. Wei, "Data organization patterns for cloud enterprise applications," in *Proc. Asia-Pacific Services Comput. Conf. (APSCC)*, 2014, pp. 1–7.
- [13] U.S. Department of Health & Human Services. (1996). *Health Insurance Portability and Accountability Act of 1996 (HIPAA)*. [Online]. Available: <https://www.hhs.gov/hipaa/>
- [14] European Parliament, *General Data Protection Regulation (GDPR)*, document Regulation (EU) 2016/679, Apr. 2016. [Online]. Available: <https://gdpr-info.eu/>
- [15] S. Zeng, J. Zhang, P. He, Y. Liu, Y. Xing, H. Xu, J. Ren, Y. Chang, S. Wang, D. Yin, and J. Tang, "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in *Proc. Findings Assoc. Comput. Linguistics ACL*, Aug. 2024, pp. 4505–4524.
- [16] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur.*, Copenhagen, Denmark, Nov. 2023, pp. 79–90, doi: [10.1145/3605764.3623985](https://doi.org/10.1145/3605764.3623985).
- [17] L. Ammann, S. Ott, C. R. Landolt, and M. P. Lehmann, "Securing RAG: A risk assessment and mitigation framework," 2025, *arXiv:2505.08728*.
- [18] B. Chen, J. Tackman, M. Setälä, T. Poranen, and Z. Zhang, "Integrating access control with retrieval-augmented generation: A proof of concept for managing sensitive patient profiles," in *Proc. 40th ACM SIGAPP Symp. Appl. Comput.*, Mar. 2025, pp. 915–919.

- [19] M. Stähler, F. Köster, T. Müller, and C. Langdon, “VAULT: Verified access control for LLM-based knowledge graph querying,” in *Proc. 17th Int. Conf. Inf., Process, Knowl. Manage. (eKNOW)*, 2025, pp. 21–29.
- [20] E. Jiang, A. Chen, I. Tenison, and L. Kagal, “MediRAG: Secure question answering for healthcare data,” in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2024, pp. 6476–6485.
- [21] M. Aldosary and N. Alqahtani, “A survey on federated identity management systems: Limitations and solutions,” *Int. J. Netw. Secur. Appl. (IJNSA)*, vol. 13, no. 4, pp. 1–11, Jul. 2021.
- [22] A. Bumiller, E. Kozyri, and H. Dagenborg, “Privacy preserving identity federation: A literature study,” *ACM Comput. Surv.*, Jul. 2025, doi: [10.1145/3745018](https://doi.org/10.1145/3745018).
- [23] M. R. Asghar, M. Backes, and M. Simeonovski, “PRIMA: Privacy-preserving identity and access management at internet-scale,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2018, pp. 1–6.
- [24] D. Pöhn and W. Hommel, “Reference service model framework for identity management,” *IEEE Access*, vol. 10, pp. 120984–121009, 2022, doi: [10.1109/ACCESS.2022.3219044](https://doi.org/10.1109/ACCESS.2022.3219044).
- [25] J. Anderson and K. Keahey, “Migrating towards single sign-on and federated identity,” in *Proc. Pract. Exper. Adv. Res. Comput.*, Boston, MA, USA, Jul. 2022, pp. 1–8, doi: [10.1145/3491418.3530770](https://doi.org/10.1145/3491418.3530770).
- [26] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.
- [27] S. Cantor. (Mar. 2005). *Assertions and Protocols for the OASIS Security Assertion Markup Language (SAML) V2.0*. OASIS Standard. [Online]. Available: <http://docs.oasis-open.org/security/saml/v2.0/saml-core-2.0-os.pdf>
- [28] D. Hardt, Ed., *The OAuth 2.0 Authorization Framework*, document RFC 6749, IETF, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/info/rfc6749>
- [29] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore. (Nov. 2014). *OpenID Connect Core 1.0 Incorporating Errata Set 1*. OpenID Foundation. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html
- [30] AWS. *Temporary Security Credentials in IAM*. AWS Identity Access Manage. User Guide. Accessed: Nov. 10, 2025. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html
- [31] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Brussels, Belgium, Oct. 2018, pp. 2369–2380.
- [32] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (Short Papers)*, vol. 2, Melbourne, VIC, Australia, Jul. 2018, pp. 784–789, doi: [10.18653/v1/p18-2124](https://doi.org/10.18653/v1/p18-2124).
- [33] Y. Mao, X. Dong, W. Xu, Y. Gao, B. Wei, and Y. Zhang, “FIT-RAG: Black-box RAG with factual information and token reduction,” *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–27, Mar. 2025.
- [34] W. Yu, D. Iter, S. Wang, Y. Xu, M. Ju, S. Sanyal, C. Zhu, M. Zeng, and M. Jiang, “Generate rather than retrieve: Large language models are strong context generators,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Kigali, Rwanda, May 2023, pp. 1–27.
- [35] J. Jeong and S.-G. Lee. (2025). *Permission-Aware RAG: Identity Access Manage. (IAM)-Based Access Filtering Multi-Resource Environments—Code Repository*. GitHub. [Online]. Available: <https://github.com/JooyoungJeong/permission-aware-rag>



JOOYOUNG JEONG received the B.S. degree in computer science from Sangmyung University, Seoul, South Korea, in 2012. He is currently pursuing the Master of Engineering degree with the Graduate School of Engineering Practice, Seoul National University, Seoul. He is a Software Engineer at Samsung Electronics. His research interests include retrieval-augmented generation (RAG), AI security and privacy, recommender systems, and large language models.



SANG-GOO LEE (Member, IEEE) received the B.S. degree in computer science and statistics from Seoul National University, South Korea, in 1985, and the M.S. and Ph.D. degrees in computer science from Northwestern University, Evanston, IL, USA, in 1987 and 1990, respectively. Since 2014, he has been the Associate Director of the Big Data Institute, Seoul National University, where he is currently a Professor with the Department of Computer Science and Engineering. His research interests include database systems, deep learning, natural language processing, and recommender systems.

...